

Entitlement Bypass via AI-Agent-Generated Archives Served Through Unauthenticated Public File URLs

Product	Base44 (base44.com / base44.app) — AI application builder, acquired by Wix
Report type	Business-logic / entitlement bypass with an unauthenticated-disclosure component
Severity	Medium — CVSS 3.1: AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N (~5.3); upgrade path to High pending vendor confirmation of file-serving authorization model
Reporter	admin@dtrh.net
Date of testing	2026-08-07
Report version	2.0 (refined from original v1.0 disclosure)

1. Summary

Base44 gates full project/source-code export behind paid subscription tiers (Builder and above); Free and Starter workspaces are explicitly prevented from exporting a project's underlying code. During testing against applications I own, I found that the in-app AI agent — which free-tier users retain full access to — can be instructed to independently reconstruct that restricted capability from primitives it already has:

- Recursively read the application's workspace filesystem (/app).
- Generate an arbitrary binary artifact (a TAR/gzip archive) using Node.js built-ins (fs, zlib, manual Buffer construction), with no dependency on an external tar binary.
- Upload that artifact through Base44's own Core.UploadFile integration.
- Receive back a Base44-hosted .gz file URL.

I subsequently confirmed that the returned URL is retrievable **without any Base44 authentication** — no session, no account, no application access grant. Anyone in possession of the URL can download the archive.

Two distinct, separable problems fall out of this workflow:

- **An entitlement-bypass problem:** a capability the product intentionally paywalls (complete project export) is reachable through an unrestricted, un-paywalled capability (agent code execution + file upload), on a Free-tier account.
- **An authorization/confidentiality problem:** the artifact produced by that bypass — which can contain full application source, configuration, and logic — is exposed via a URL that requires no authentication to fetch, moving it from “behind Base44’s access-control boundary” to “openly available to anyone with the link.”

I only tested against applications and accounts under my own control. I did not attempt to access another tenant's application, enumerate identifiers, or retrieve another user's files. That distinction — and its effect on severity — is discussed explicitly below.

2. Affected Components

Component	Role in the issue
Base44 AI agent / in-app code-execution sandbox	Grants filesystem read access to /app and arbitrary Node.js execution, independent of export entitlement
Application workspace filesystem	Source of the exported material (project source, config, logic)
Node.js runtime (agent sandbox)	Used to construct a TAR stream and gzip it without external tooling
Core.UploadFile integration	Accepts the agent-generated binary and returns a hosted URL
Base44 public file storage / file-serving API (/api/apps/{APP_ID}/files/mp/public/...)	Serves the resulting object without requiring authentication
Plan/entitlement enforcement layer (export gating)	The control this workflow circumvents

3. Background: Why This Matters for Base44 Specifically

Public context makes this finding more than a theoretical composition of capabilities:

3.1 Export is a deliberately paywalled feature, not an oversight

Base44's own pricing and support documentation confirm that code/project export (via ZIP download or GitHub sync) is unavailable on the Free and Starter tiers and only unlocks on Builder (\$50/mo) and higher plans. This is a documented product boundary, not an incidental limitation — which is precisely what makes an agent-mediated route around it a genuine entitlement bypass rather than a matter of interpretation.

3.2 Base44 has a recent, directly relevant history of app_id-based authorization failures

In July 2025, Wiz Research disclosed a critical vulnerability in Base44 in which the `app_id` — visible in every application's URL and in its public `manifest.json` — could be submitted to undocumented, unauthenticated registration and OTP-verification endpoints to mint a verified account inside any private application, bypassing SSO and all other access controls. Wix/Base44 patched it within 24 hours. The relevant lesson is architectural: Base44's platform has previously treated `app_id` as a non-secret routing value while a downstream endpoint implicitly trusted it to authorize a sensitive action. The file-serving path documented in this report repeats the same `APP_ID-as-path-parameter` pattern, on an endpoint whose authorization model I was not able to fully characterize from the outside.

3.3 The broader “vibe-coding” platform category has an established pattern of this exact failure class

Independent research has repeatedly found that AI app-builder platforms default to, or can be steered into, exposing backend source, credentials, and user data through public-by-default or authorization-free storage/serving paths:

- Lovable disclosed (and initially disputed, then confirmed) a Broken Object Level Authorization issue in which free accounts could read other users' project source code, database credentials, and AI chat histories (CVE-2025-48757).
- Independent testing of a single Lovable-hosted application found 16 distinct vulnerabilities, including exposure of over 18,000 users' data, rooted in the Supabase backend that many vibe-coded apps share without platform-enforced RLS.
- RedAccess reported finding roughly 380,000 publicly accessible assets built with Lovable, Base44, Replit, and Netlify — including thousands exposing sensitive corporate data — largely because privacy/visibility defaults make apps public unless a user manually opts out, and because AI agents/generated code frequently omit access controls a human developer would add.

This report's finding is a different mechanism (agent-mediated file upload rather than a registration endpoint or missing RLS), but it belongs to the same overall failure category: capabilities that are individually reasonable — agent filesystem access, agent code execution, a “public file” upload primitive — compose into an unintended, unauthorized export/disclosure channel when no layer in the chain re-checks authorization.

4. Technical Details

4.1 Archive construction (agent side, no export entitlement required)

The agent, operating from the application workspace root (`/app`), was instructed to build a project archive without using an OS-level `tar` binary or an external archiving package — entirely through Node.js built-ins:

```
const fs = require("fs");
const path = require("path");
const zlib = require("zlib");
```

A TAR (USTAR) header and body were constructed manually using `Buffer` objects, e.g.:

```
const header = Buffer.alloc(512, 0);
header.write("ustar\0", 257, "ascii");
```

The resulting TAR byte stream was piped through Node's built-in `zlib.gzip`. Conceptually:

```
/app
→ recursive filesystem traversal
→ TAR byte stream (hand-built USTAR headers + file contents)
→ zlib gzip
→ .gz archive
```

No external `tar` executable, and no dependency outside Node's standard library, was required. This matters because the archive-construction step cannot be blocked by restricting which binaries or packages are available in the agent's execution environment — it only requires that the agent retain (a) filesystem read access and (b) a general-purpose scripting/code-execution capability, both of which are core, non-paywalled features of the product.

4.2 Upload via `Core.UploadFile`

The resulting binary was passed to Base44's built-in upload integration:

```
await base44.integrations.Core.UploadFile({
  file: archive
});
```

Where a Blob representation was needed:

```
const blob = new Blob([gzBuffer], {
  type: "application/gzip"
});
```

The call returns a Base44-hosted file URL of the form:

```
https://base44.app/api/apps/{APP_ID}/files/mp/public/{APP_ID}/{REF_ID}_{FILENAME}.gz
```

4.3 Confirmed unauthenticated retrieval

I verified that the returned URL can be fetched from a browser session with no active Base44 authentication. A second, independently generated archive was confirmed retrievable this way:

```
https://base44.app/api/apps/6a76d4c4f175cfa6f22f29f0/files/mp/public/
6a76d4c4f175cfa6f22f29f0/66a051d1e_minutementortar.gz
```

The observed access model is:

Possession of URL → HTTP request → Base44 file service → Archive returned

rather than:

Possession of URL → Authentication → Application-level authorization → Archive returned

I did not test whether the identifiers in the URL are intended to function as bearer-style secrets, and I did not attempt to guess or enumerate them.

4.4 URL structure and identifier interpretation

The path contains the application identifier twice:

```
/api/apps/{APP_ID}/files/mp/public/{APP_ID}/{REF_ID}_{FILENAME}.gz
```

Field	Example value	Notes
APP_ID	6a76d4c4f175cfa6f22f29f0	24-character hex; matches standard MongoDB ObjectId format (4-byte timestamp + 5-byte random/machine + 3-byte counter)
REF_ID	66a051d1e	9 hex characters; distinct per uploaded object, appears to identify the file rather than the application
FILENAME	minutementortar	Derived from the archive's original filename

Two independently generated archives had matching APP_ID values but different REF_ID values, supporting the interpretation that REF_ID scopes the individual uploaded object while APP_ID scopes the owning application.

Reconstructing the full path segment by segment makes the trust boundary easier to reason about:

base44.app	platform host
/api	platform API root
/apps	app-scoped resource namespace
/ {APP_ID} (1st)	request scope – “this call is for app X”
/files	uploaded-file storage route
/mp	storage mount / bucket prefix
/public	authless tier (counterpart: signed-URL private tier)
/ {APP_ID} (2nd)	storage namespace – “this file's owner folder”
/ {REF_ID}_{name}	stored object – dedup/collision prefix + original filename

#	Segment	Role	Confidence
1	base44.app	Platform host serving all platform APIs and hosted assets	Confirmed
2	/api	Platform API root — distinguishes platform endpoints from routes inside a published app	Confirmed
3	/apps	App-scoped resource namespace; signals that the next segment is an app identifier	Confirmed

#	Segment	Role	Confidence
4	/[APP_ID] (1st)	Scopes the request to a specific application — the routing/authorization parameter	Confirmed
5	/files	Uploaded-file storage route (objects written via UploadFile / UploadPrivateFile)	Confirmed
6	/mp	Storage mount/bucket prefix separating the blob layer from the logical /files route	Inferred
7	/public	Authless storage tier — files here require no authentication. Implies a counterpart private tier	Confirmed behaviorally
8	/[APP_ID] (2nd)	Per-app owner folder inside public storage — keeps different applications' public files from colliding	Confirmed
9	/[REF_ID]_{FILENAME}	Stored object name: short hex prefix for collision-avoidance/dedup + original filename	Confirmed structurally

The identifier appears twice for two distinct reasons, not redundantly: the first occurrence answers “which application is this request authorized against?”, and the second answers “which application’s folder does this object physically live in?” Both currently resolve from the same untrusted, attacker-suppliable value, which is the crux of the authorization question raised in §9, Scenario C.

4.4.1 APP_ID timestamp decoding

Because APP_ID follows the MongoDB ObjectID layout, its first 8 hex characters decode directly to a Unix timestamp (seconds since epoch). Decoding the two APP_ID values observed during testing:

APP_ID	Timestamp segment	Decoded creation time (UTC)
6a76d4c4f175cfa6f22f29f0	6a76d4c4	2026-08-08 07:03:32
6a76395db2ecc5051b824952	6a76395d	2026-08-07 20:00:29

Both values decode to timestamps consistent with when the corresponding test applications were actually created. The practical implication is that APP_ID is not just non-secret (per the July 2025 Wiz finding) — it is also coarsely predictable in time. An attacker who learns the approximate creation time of a target application has already narrowed the timestamp component to a window of hours, leaving only the 8-byte random/counter suffix to search. This does not by itself demonstrate a practical attack (the random/counter portion is still 64 bits), but it is a relevant input to how Base44 should weigh the residual risk of treating APP_ID as path-scoping rather than as an access-control credential.

4.4.2 REF_ID composition and entropy

The 9-hex-character REF_ID prefix represents 36 bits of keyspace — approximately 6.87×10^{11} (68.7 billion) possible values per stored object. Two plausible generation methods would produce a token of this shape:

- **Truncated content hash** (e.g., first 9 hex characters of a SHA-1/SHA-256 digest). This would make REF_ID a deterministic function of file content — identical bytes uploaded twice would always produce the same object reference. A side effect is that if an attacker already knows the exact bytes of a file of interest, they can compute its REF_ID directly.
- **Randomly generated token** (e.g., `crypto.randomBytes`) used purely to prevent same-named uploads from colliding. This is the more defensible design if REF_ID is intended to function as a bearer-style access credential, but its real-world guessing resistance still depends entirely on Base44’s rate-limiting at the file-serving endpoint.

I was not able to distinguish between these two hypotheses from external testing, and I did not attempt to enumerate or brute-force any REF_ID value — doing so was out of scope. I flag the distinction because it directly determines how much weight REF_ID unpredictability alone can bear as a security control.

4.5 Follow-up test: public HTML workspace index

In a second test, the agent was instructed to produce a single self-contained HTML index of the workspace rather than one archive. The output:

- Recursively walked `/app`.
- Inlined the contents of 45 small text files directly into the HTML (visible via `<details>/<pre>` blocks — no upload needed for these).
- Uploaded 9 larger files individually via `Core.UploadFile`.

- Produced one HTML page listing all 54 files, with the 9 uploaded ones as clickable links to their public URLs, and uploaded the index itself the same way.

The page self-reported: **54 files · 9 uploaded · 45 embedded**. All uploaded objects — including the index page — were served under the same public path structure. This confirms the archive-upload behavior generalizes: **any individual source file** the agent can read is independently uploadable and independently retrievable without authentication, not just a bundled archive.

4.6 Observed agent behavior (representative)

When prompted to attempt the archive/upload workflow, the agent narrated its own actions in-line: it confirmed it could reach the sandbox filesystem at /app, proposed hand-rolling a USTAR archive excluding node_modules and .git, and reported success with a file count and compressed size once the upload completed. This indicates the agent treated the workflow as a normal, unremarkable use of its available tools — there was no refusal, warning, or entitlement check surfaced to the user at any point in the chain.

5. Reproduction Steps

Performed only against applications and accounts under my control:

1. Create or open a Base44 application on a Free-tier workspace (export/download disabled by plan).
2. In the agent chat, request that it inspect and archive the project workspace.
3. Instruct it to build the archive using only Node.js built-ins (fs, zlib, manual Buffer/TAR-header construction) rather than an external tar binary or npm package.
4. Instruct it to gzip the resulting TAR stream.
5. Instruct it to upload the resulting binary via Core.UploadFile.
6. Record the returned Base44 file URL.
7. Open the URL in a private/incognito browser session with no active Base44 login.
8. Observe that the archive downloads successfully with no authentication challenge.

Resulting URL pattern:

`https://base44.app/api/apps/{APP_ID}/files/mp/public/{APP_ID}/{REF_ID}_{FILENAME}.gz`

6. Expected vs. Actual Behavior

Expected

If complete project export is a paid-tier entitlement, and if application source is intended to remain behind Base44's authentication/authorization boundary, then no combination of Free-tier-available agent capabilities should be able to reconstruct a full-project export and place it somewhere retrievable without authentication. At minimum, an artifact that is functionally equivalent to a project export (regardless of how it was produced) should inherit the same authorization requirements as the export feature itself.

Actual

A Free-tier agent was able to (1) read the full application workspace, (2) synthesize a complete project archive without needing the gated export feature, (3) upload it through a general-purpose file-upload integration, and (4) obtain a URL that serves the archive to anyone, unauthenticated. The archive exists entirely outside both the plan-entitlement boundary and the application's normal access-control boundary.

7. Security Boundary Analysis

The core issue is not any single capability — it is their composition:

```
Workspace filesystem access
+
Arbitrary binary/code generation (agent sandbox)
+
General-purpose public file upload (Core.UploadFile)
+
Unauthenticated object retrieval
=
An alternative, unpaywalled, unauthenticated project-export channel
```

Each capability is individually defensible as a legitimate product feature: agents need workspace read access to do their job; agents need general code-execution to build features; `Core.UploadFile` is a documented way for an app (or its agent) to store and share a file publicly. **None of these three, alone, is a vulnerability.** The vulnerability is that no layer in the chain re-validates the more sensitive question: “should this specific content — a full snapshot of the application’s own source — be allowed to leave the authorization boundary via this specific mechanism?”

A file-upload primitive designed for arbitrary user-generated content is not content-aware, and the agent sandbox is not export-entitlement-aware. The result is a classic **confused-deputy / excessive-agency pattern**: a component with legitimate authority is induced to perform an action that the platform’s own product logic says the user should not currently be entitled to perform directly. This class of failure is increasingly well-documented for agentic systems — OWASP’s LLM application risk guidance names “Excessive Agency” as a top-tier concern precisely because agents are frequently granted broad, composable tool access without corresponding fine-grained, action-level authorization checks.

8. Confidentiality Impact

The unauthenticated nature of the resulting URL is the component that most increases real-world impact. An attacker in possession of a generated URL needs none of the following: a Base44 account; access to the target’s dashboard; an authenticated session; permission to use the underlying application; or access to the development/agent environment. They need only the URL itself.

If that URL — or the APP_ID/REF_ID pair needed to construct it — is exposed through any secondary channel (client-side JavaScript, a shared link, a referrer header, a log line, an indexed public preview, a chat transcript pasted somewhere), the archive is retrievable by anyone, indefinitely, with no logging visible to the application owner and no way to revoke access short of the platform deleting the object. This is materially the same shape of risk that RedAccess’s 2026 scan surfaced across Base44, Lovable, Replit, and Netlify at scale.

9. Security Impact Assessment — Scenarios

Severity is genuinely conditional on facts only Base44 can confirm about intended behavior. Three scenarios, in increasing order of severity:

Scenario A — Product/plan-entitlement bypass only. If the export restriction exists purely as a monetization control over a user’s own data (i.e., the user is already fully authorized to possess this content, and export is simply gated to encourage upgrades), then the impact is a revenue/entitlement-bypass issue. This alone is a legitimate, reportable finding, but its confidentiality impact is limited to the reporter’s own data.

Scenario B — Export restriction is a data-protection control, not just monetization. If Base44 intends the export restriction to also limit a user’s ability to extract complete backend source/logic (for platform-integrity, IP, or dependency reasons), then bypassing it via agent tool composition is a more substantive authorization/control failure, independent of the public-URL issue.

Scenario C — The public file-serving path itself lacks authorization, beyond just this workflow. If `/api/apps/{APP_ID}/files/mp/public/...` performs no check that the requester is authorized to access the owning application — only that the URL/identifiers are well-formed — then this is a confidentiality/access-control issue at the file-serving layer itself, of which the agent-archive workflow is only one way to populate it. Given Base44’s confirmed prior history of app_id-scoped endpoints trusting that identifier as sufficient for authorization (§3.2), this scenario cannot be ruled out from external testing alone and warrants direct verification by the vendor.

I did not test Scenario C against another tenant’s application or attempt to determine whether APP_ID/REF_ID values are guessable, enumerable, or leaked through any secondary channel. The severity rating below reflects Scenarios A/B as confirmed and treats Scenario C as an open question requiring vendor-side verification, not a confirmed finding.

10. Severity Justification

Self-assessed: Medium, using a CVSS 3.1 base vector of `AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N` (≈ 5.3):

- **Attack Vector: Network** — reachable entirely over the web.
- **Attack Complexity: Low** — reproducible via natural-language prompting of the standard agent interface; no exploit development needed.

- **Privileges Required: None** (from the perspective of the person retrieving the resulting URL) / a Free-tier account is sufficient to produce the archive.
- **User Interaction: None** to retrieve an already-generated URL.
- **Confidentiality: Low** — confirmed only for the reporter's own application content; not confirmed as generalizable to arbitrary tenants.
- **Integrity / Availability: None** — no modification or disruption capability was identified or tested.

This rating reflects what was **confirmed**: an entitlement bypass, plus unauthenticated retrieval of self-generated content. It does not assume Scenario C. **If Base44 confirms that the file-serving endpoint performs no per-application authorization check** (i.e., that any correctly-formed URL for any application is retrievable by anyone, and that APP_ID/REF_ID values are not effectively unguessable secrets), the Confidentiality component should be revised upward — plausibly to High, given Base44's enterprise customer base and documented history of hosting HR systems, internal knowledge bases, and other PII-bearing applications. That determination requires internal visibility Base44 has and I do not.

11. Suggested Remediation

Ordered by estimated impact-to-effort ratio:

1. Treat Core.UploadFile output as sensitive-by-default when the source is agent-generated from the workspace itself

Distinguish “user uploaded this external file” from “the agent read and repackaged the project's own source” — the latter should not be eligible for the same public-by-default serving path.

2. Enforce export/plan entitlement checks at the point of content generation or upload, not only at the dedicated Export UI

If full-project export is a paid feature, any code path that can produce a substantially equivalent artifact should be subject to the same entitlement check, regardless of which tool triggered it.

3. Require authentication and application-level authorization on file retrieval for any object that contains, or was derived from, application source/configuration

Even if the object was placed in “public” storage. Reserve genuinely unauthenticated public storage for content explicitly intended for anonymous public consumption.

4. Add content classification/heuristics on agent-initiated uploads

Flag or block uploads that closely mirror the shape of the project workspace (many files, source-code MIME types/extensions, filenames matching workspace paths) for additional review or a stricter authorization tier.

5. Rate-limit and audit agent-initiated Core.UploadFile calls

Particularly on Free/Starter tiers, and particularly for uploads generated from filesystem-traversal-style agent actions rather than single discrete outputs.

6. Reconsider using the same APP_ID as both a path-scoping value and an implicit authorization signal

Given the July 2025 app_id-trust incident and the fact that APP_ID's ObjectID structure makes its creation time directly decodable, any endpoint that accepts APP_ID in the URL should explicitly re-verify the requester's authorization to that application.

7. Confirm and document REF_ID generation strength

If REF_ID is meant to function as an unguessable capability token, ensure sufficient entropy beyond the ~36 bits observed, confirm it is not derived from file content, and apply request-rate limiting. If it is not meant to function this way, authentication must not be optional at the serving layer.

8. Extend product documentation

State plainly whether files placed via Core.UploadFile are permanently and unconditionally public, so that both end users and agents can treat that capability accordingly. Consider adding an explicit agent-level guardrail preventing bulk workspace-to-Core.UploadFile operations without user confirmation of the public-exposure implications.

12. Evidence Available on Request

- Original test applications used for reproduction.
- Exact prompt/instruction sequence given to the agent.

- Node.js archive-generation code as generated by the agent.
- Generated archives (.gz) and generated HTML index.
- Returned Base44 file URLs.
- Screen recordings/logs demonstrating unauthenticated retrieval.
- A recovered GitHub repository containing the first generated archive:
<https://github.com/DTRHnet/markov-qualia-consciousness-simulator>
- Relevant Base44-generated client/integration code referencing Core.UploadFile.

Confirmed unauthenticated archive example (self-owned application):

```
https://base44.app/api/apps/6a76d4c4f175cfa6f22f29f0/files/mp/public/6a76d4c4f175cfa6f22f29f0/66a051d1e_minutementortar.gz
```

13. Scope Limitations and Responsible Disclosure Statement

All testing described in this report was performed exclusively against applications and accounts I own. I explicitly did **not**:

- access another user's application or account;
- attempt to enumerate APP_ID or REF_ID values;
- attempt to guess or brute-force any identifier;
- retrieve, modify, or interact with another user's files or application;
- perform denial-of-service or load-testing activity;
- exploit the confirmed behavior beyond what was necessary to document and reproduce it.

I am not claiming — and this report should not be read as claiming — that cross-tenant access has been demonstrated. Scenario C (§9) is explicitly flagged as an open question for Base44's security team to evaluate with internal visibility into the file-serving authorization model, not as a confirmed finding of this report.

I am submitting this privately so Base44 can determine whether the documented behavior reflects an intended capability, a plan-entitlement gap, or an authorization defect, and I am available to provide any of the evidence listed in §12 to support triage.

Research Notes

The following public sources informed the contextual and remediation sections of this refined report. No claims about Base44's internal implementation were made or inferred beyond what was directly observed in testing; these sources were used solely to (a) confirm that project export is a documented, paid-tier-gated feature, (b) establish precedent for app_id-related authorization issues specific to Base44, (c) situate this finding within the broader, independently documented pattern of authorization failures across AI app-builder ("vibe coding") platforms, and (d) ground the remediation recommendations in current sandboxing/agent-authorization best practice.

- Wiz Research — original disclosure of the July 2025 Base44 app_id-based authentication bypass, and subsequent press coverage confirming the vulnerability, its root cause, and its 24-hour remediation timeline.
- Base44 pricing and third-party plan-comparison sources confirming that code/project export is unavailable on Free and Starter tiers and gated to Builder-and-above paid plans.
- Reporting on Lovable's 2026 authorization incidents documenting a Broken Object Level Authorization vulnerability (CVE-2025-48757) and related findings, used here as category precedent.
- Axios reporting on RedAccess's 2026 research identifying approximately 380,000 publicly accessible assets across Base44, Lovable, Replit, and Netlify.
- General AI-agent-security literature (OWASP LLM application risk guidance on "Excessive Agency"; academic and industry material on AI coding-agent sandboxing and confused-deputy-style failures).
- The MongoDB ObjectID specification (public, vendor-agnostic 12-byte identifier format) was used to interpret and decode the APP_ID values observed in testing. The specific observation that Base44's APP_ID values conform to this format, and the resulting decoded timestamps, come from this report's own testing.

This refined report (v2.0) preserves every factual claim from the original disclosure while substantially expanding contextual research, technical analysis of identifier structure and entropy, security-boundary reasoning, and prioritized remediation guidance. It is intended for responsible disclosure to the Base44 / Wix security team.